

```

1  #include <stdio.h>
2
3  #define MAX_N 100
4  #define NO_EDGE -1
5  #define EMPTY -1
6
7  typedef struct {
8      int n;
9      int M[MAX_N][MAX_N];
10 } Graph;
11
12 void init_graph(Graph *G, int n) {
13     G->n = n;
14     for (int i = 0; i <= n; i++) {
15         for (int j = 0; j <= n; j++) {
16             G->M[i][j] = NO_EDGE;
17         }
18     }
19 }
20
21 void add_edge(Graph *G, int u, int v) {
22     if (u < 0 || u > G->n || v < 0 || v > G->n || u == v) return;
23
24     G->M[u][v] = 1;
25     G->M[v][u] = 1;
26 }
27
28 void neighbours(Graph G, int u, int adjlist[]) {
29     int k = 0;
30     for (int v = 1; v <= G.n; v++) {
31         if (G.M[u][v] != NO_EDGE) {
32             adjlist[k] = v;
33             k++;
34         }
35     }
36     adjlist[k] = EMPTY;
37 }

```

```

39  #define WHITE 0
40  #define GRAY 1
41  #define BLACK 2
42
43  void depth_first_search(Graph G, int u, int parent, int color[], int *res) {
44      color[u] = GRAY; // dang duyet
45
46      int list[G.n + 1], v, i = 0;
47      neighbours(G, u, list);
48      while ((v = list[i]) != EMPTY) {
49          if (color[v] == WHITE) { // chua duyet
50              depth_first_search(G, v, u, color, res);
51          }
52          else if (v != parent && color[v] == GRAY) {
53              *res = 1;
54              return;
55          }
56          i++;
57      }
58
59      color[u] = BLACK; // da duyet
60  }
61
62  int main() {
63      int n, m, u, v;
64      scanf("%d%d", &n, &m);
65
66      Graph G;
67      init_graph(&G, n);
68      for (int i = 0; i < m; i++) {
69          scanf("%d%d", &u, &v);
70          add_edge(&G, u, v);
71      }
72
73      int color[G.n + 1], flag = 0;
74      for(int i = 0; i <= G.n; i++) {
75          color[i] = WHITE;
76      }
77      for (int u = 1; u <= G.n && !flag; u++) {
78          if (color[u] == WHITE) {
79              depth_first_search(G, u, 0, color, &flag);
80          }
81      }
82
83      if (flag) puts("YES");
84      else puts("NO");
85
86      return 0;
87  }
88

```